

Date- 05-Dec-2023

DATA TYPE AND VARIABLES

Date ____ / ____ / ____

Saathi

First program

```
#include <iostream>
int main() {
    std::cout << "Namaste Duniya!\n";
}
```

→ To write.

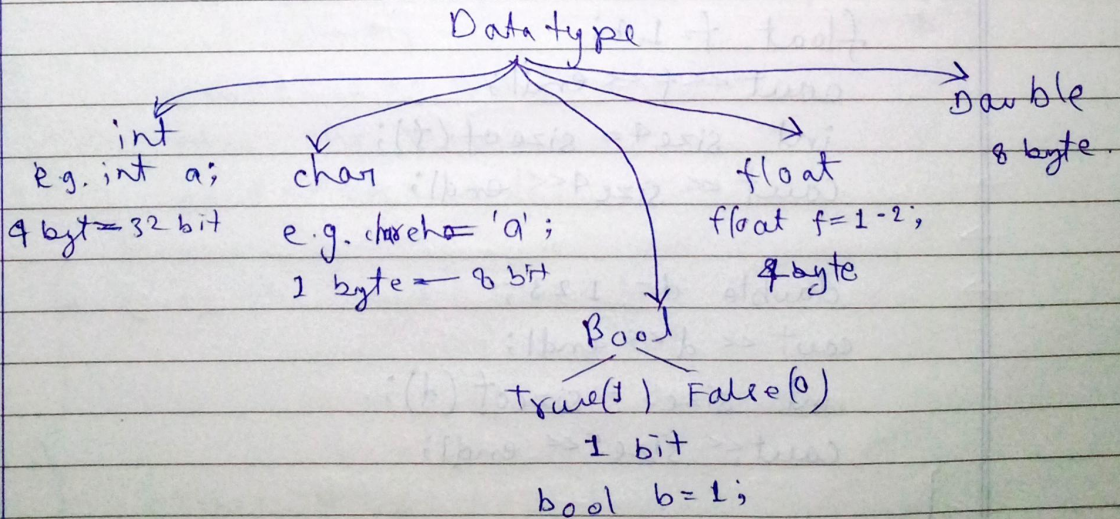
```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello World! << end;
}
```

Diagram illustrating the output of the program:

- The text "Hello World!" is printed on the first line.
- The text "end;" is printed on the second line, indicating a new line.

Namespace:

- Namespace provide the space where we can define or declare identifier i.e. variable, method, class.



Date ___/___/___

Saathi

```
#include <iostream>
using namespace std;
int main() {
    int a = 123;
    cout << a << endl;
    int size1 = sizeof(a);
    cout << size1 << endl;
```

```
    char b = 'v';
    cout << b << endl;
    int size2 = sizeof(b);
    cout << size2 << endl;
```

```
    bool bl = true;
    cout << bl << endl;
    int size3 = sizeof(bl);
    cout << size3 << endl;
```

```
    float f = 1.4;
    cout << f << endl;
    int size4 = sizeof(f);
    cout << size4 << endl;
```

```
    double d = 1.23;
    cout << d << endl;
    int size5 = sizeof(d);
    cout << size5 << endl;
```

```
}
```

output

123

4

v

1

1

1

1.4

4

1.23

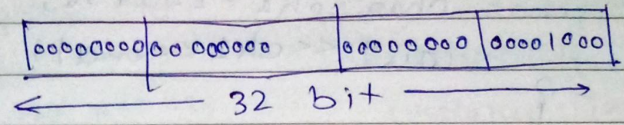
8

Date ____/____/____

How data is stored:

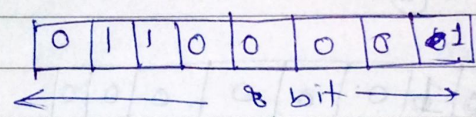
int a = 8
 ↓
 4 byte

in binary 1000



char ch = 'a';
 ↓
 1 byte

ASCII value = 97



Binary

Type casting

```
#include <iostream>
using namespace std;
int main() {
    int a = 'a';
    cout << a << endl;
    char ch = 98;
    cout << ch << endl;
}
```

output: 97
 b

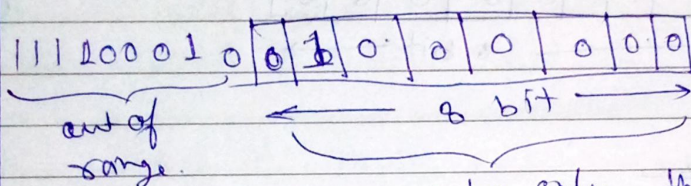
- int → 4 byte → 32 bit
 max size = $2^{32}-1$
 min size = 0
- char → 1 byte → 8 bit
 max size = 2^8-1
 min size = 0

what happens if $ch > 2^8-1$
 and print it

Date ___ / ___ / ___

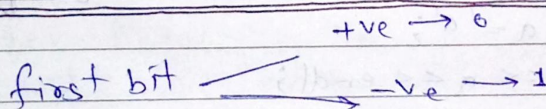
```
#include <iostream>
using namespace std;
int main() {
    char ch1 = 123456;
    cout << ch1 << endl;
}
```

Output: warning: character value from
123456 to 64
@



take only this value
the value is 64 = @

How -ve number are stored:



for display
↓
take 2's
complement

- ignore the -ve sign.
- convert it into binary representation
- take 2's complement and store

example 2's complement

(000101)

1's complement - 01110100

2's complement

@1110101

↑
+ve number

range -2^{31} to $+2^{31}-1$

unsigned int range 0 to $2^{32}-1$

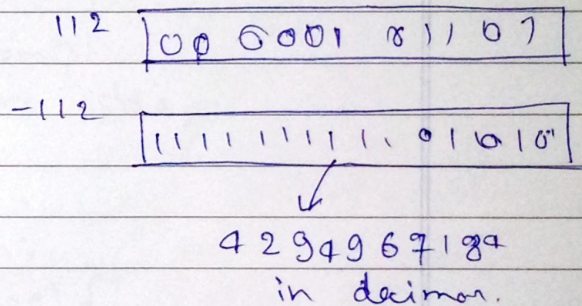
```
#include <iostream>
using namespace std;
int main() {
    unsigned int a2 = 112;
    cout << a2 << endl;
}
```

output: 112

```
#include <iostream>
using namespace std;
int main() {
    unsigned int a2 = -112;
    cout << a2 << endl;
}
```

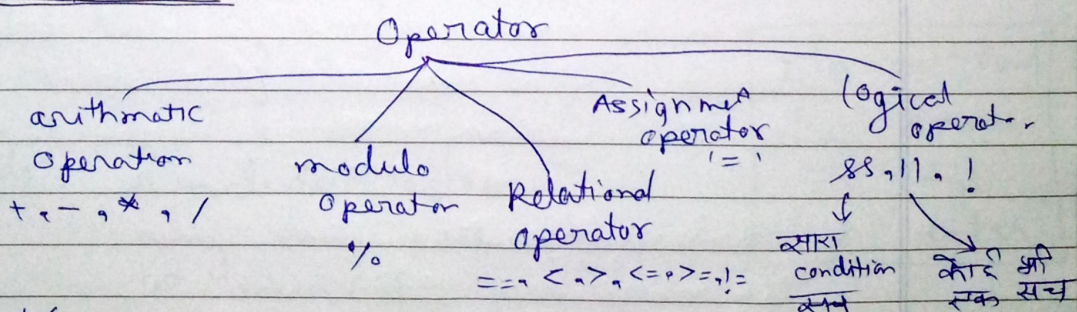
output: 4294967184

↓
output due to unsigned int



for resolve this error we use signed.

Operators:



int/int = int

float/int = float

double/int = double